

Popular Vote

Technical Design Document

Date Updated: 2026/03/17

Version:4.0

Team: Alou Kone, Zachery Francis, Jonathan Koral, Andrew Li

Context

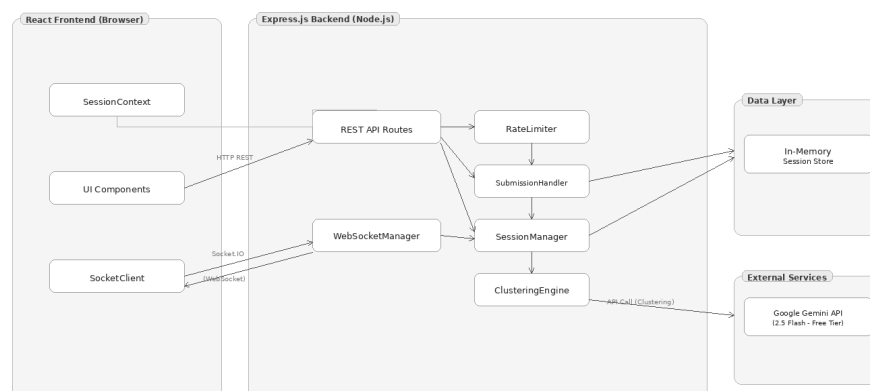
This document describes the technical design for Popular Vote, a web-based application that allows participants in a shared session to anonymously submit questions about a topic. The system collects these questions, uses AI to cluster similar ones, and condenses them into a clear, organized set of representative queries for the session host. The requirements document for this project can be found in the team's shared drive.

The primary technical challenge for this project is threefold. First, the system must support live, concurrent question submission from many participants while maintaining anonymity, no identifying information can be attached to any submission. Second, the system must integrate with a third-party language model API to perform semantic clustering of freeform text questions in near-real-time. Third, the system must coordinate a shared session lifecycle (open, closed, clustered, ended) across a host and many participants using WebSockets, so that participants see live updates, including incoming submission counts and the clustered results, without requiring page refreshes.

To solve these problems, the application uses a React frontend communicating with a Node.js/Express backend over both REST and WebSocket connections. Session state is maintained in-memory on the server during a session's lifecycle. The AI clustering step is handled by a server-side call to the Google Gemini API (free tier). WebSocket connections push real-time updates to all connected clients.

Architecture Design

Architecture Diagram



The React frontend communicates with the Express.js backend through two channels: a REST API for discrete actions (create session, submit question, trigger clustering) and a WebSocket connection (via Socket.IO) for real-time state updates (participant count, session state transitions, clustered results broadcast). The backend holds all session state in-memory and makes outbound calls to the Google Gemini API when clustering is triggered.

Key Architectural Decisions

1. Client-server web architecture

Popular Vote is a standard client-server web application. The React frontend runs in the user's browser and communicates with a centralized Express.js backend. This is a natural fit because the system requires a shared session state visible to multiple users, and a central server is the simplest way to coordinate that state.

2. Dual communication channels: REST + WebSocket

Discrete, one-off actions (creating a session, submitting a question, triggering clustering) use standard REST endpoints. Real-time state updates (participant count changes, session state transitions, clustered results broadcast) are pushed to clients over a persistent WebSocket connection via Socket.IO. This separation keeps the REST API simple and stateless for actions that don't need live updates, while the WebSocket channel handles the event-driven, push-based communication that makes the participant and host experiences feel responsive.

3. In-memory session state

All session data lives in server memory for the duration of a session. There is no database. Sessions are ephemeral by design, once a session ends, all data is discarded. This aligns with the requirements (NFR5: no data retention after session end) and dramatically simplifies the backend. The tradeoff is that a server restart kills all active sessions, which are the accepted tradeoffs of an in-memory, session-scoped design.

4. Server-side clustering via Google Gemini API

The clustering logic runs on the server, not the client. The server collects all submitted questions, constructs a prompt, and sends it to the Google Gemini API. This keeps the API key secure (never exposed to the browser), centralizes the clustering logic, and ensures the host and participants receive identical results. Gemini's free tier provides sufficient throughput, clustering is a single API call per session, well within the 250 requests/day limit.

5. Session discovery via short codes

Participants join sessions using a short, unique alphanumeric code shared by the host. There is no session listing or discovery mechanism. This matches the real-world use case: the host projects or announces the code, and participants type it in. It also enforces session privacy (NFR2) without requiring authentication.

Technology Stack

Layer	Choice	Notes
Frontend Framework	React	Component-based UI, team familiarity
Frontend Language	JavaScript (ES6+)	Course-aligned, broad ecosystem
Styling	CSS Modules	Scoped styles, no extra dependencies
Backend Framework	Express.js	Minimal, flexible, Express handles the asynchronous demands of Socket.IO and the Gemini SDK
Backend Language	Node.js	Shared language with frontend, async-native
Real-Time Communication	Socket.IO	WebSocket abstraction with rooms + fallback
AI Clustering	Google Gemini API (2.5 Flash)	Free tier, structured JSON output
Gemini SDK	@google/genai	Official Node.js SDK
Session State	In-memory (Map)	Ephemeral by design, no persistence
Rate Limiting	express-rate-limit	Prevents submission spam (NFR7)

Justifications for each choice are documented in the Key Decision Log at the end of this document.

System Components

Backend Components

SessionManager

The SessionManager is the top-level server component responsible for the lifecycle of all active sessions. It owns the in-memory session store (a JavaScript Map keyed by session code). It handles session creation (generating unique 6-character alphanumeric codes), session lookup, and session teardown. When a session ends, the SessionManager removes all associated data from memory. It is the single point of access for all other backend components that need to read or modify session state.

SubmissionHandler

The SubmissionHandler processes incoming question submissions from participants. It validates that the submission is non-empty and within the character limit (FR3.5, FR3.6), verifies the session exists and the submission window is open (FR3.1, FR5.3), and appends the question text to the session's submission list. It also handles submission deletion (FR3.8). The SubmissionHandler strips all identifying information before storage, no IP address, device identifier, or connection ID is associated with any stored question (NFR1). After a successful submission, it emits a WebSocket event to update the host's submission count.

ClusteringEngine

The ClusteringEngine is responsible for constructing the prompt, calling the Google Gemini API via the @google/genai SDK, and parsing the structured JSON response into the internal cluster format. When the host triggers clustering, the engine receives the full list of submitted questions and any session tags the host provided. It constructs a prompt that instructs the model to group semantically similar questions, generate a representative query for each cluster, indicate approximately how many submissions contributed to each group, and list the original questions per cluster. The engine also handles multi-question submissions by instructing the model to treat them as separate questions (FR4.8). If the API call fails, the ClusteringEngine returns an error to the caller without discarding the submitted questions, allowing retry (FR4.5). The Gemini free tier allows 10 RPM and 250 RPD; more than sufficient since clustering is one call per session.

WebSocketManager

The WebSocketManager wraps Socket.IO on the server side. It manages all real-time connections, organizing them into rooms by session code. It is responsible for broadcasting session state transitions (submission window closed, clustering in progress, results available, session ended) to all clients in a session room. It also sends targeted messages to the host (e.g., updated submission count) and handles client disconnection and reconnection within a session.

RateLimiter

The RateLimiter applies per-connection throttling to the question submission endpoint, preventing any single client from flooding a session with submissions (NFR7). It uses express-rate-limit with a configurable window and max-request count. Throttled requests receive a clear error response.

Frontend Components

SessionContext

A React Context provider that holds the current session state on the client side. It stores the session code, the user's role (host or participant), the current session phase (open, closed, clustering, results, ended), and the clustered results once available. All child components read from this context rather than managing their own session state.

SocketClient

The SocketClient manages the WebSocket connection on the frontend using the socket.io-client library. It connects to the Socket.IO server using the session code as the room identifier, listens for incoming events (participant count updates, session state transitions, clustered results broadcast), and updates the SessionContext accordingly. It also exposes methods for emitting client-side events (submit question, trigger clustering, close submissions).

UI Components

The UI layer is composed of screen-level components (described in the User Experience section) and shared subcomponents. Each screen-level component reads from the SessionContext and renders the appropriate view for the current session phase and user role. Shared subcomponents include the SessionCodeDisplay (prominently shows the code and QR code for sharing), the SubmissionCounter (live count for the host), the ParticipantCounter (live connected count), the QuestionInput (text area with character counter), the SubmissionList

(participant's own questions with delete), the ClusterCard (representative query + count + expandable originals), the ProcessingIndicator (loading spinner during clustering), and the PhaseIndicator (current session phase badge).

Data Design

All data structures below are defined in JavaScript. These are the shapes of the objects held in server memory during a session's lifecycle. Nothing is persisted to disk or database.

```
// Session phases
const Phase = Object.freeze({
  OPEN: "open", CLOSED: "closed", CLUSTERING: "clustering",
  RESULTS: "results", ENDED: "ended"
});

// Core session object (server-side)
class Session {
  code;           // string - unique 6-char alphanumeric
  phase;          // Phase - current lifecycle phase
  tags;           // string[] - host-provided context tags
  hostSocketId;   // string - socket ID of the host
  participantCount; // number - live connected count
  submissions;    // Submission[] - all submitted questions
  clusters;       // Cluster[] - populated after clustering
  createdAt;      // Date
}

// Individual question submission
class Submission {
  id;           // string - server-generated UUID
  text;         // string - question text (max 500 chars)
  socketId;     // string - for deletion auth only, never logged
  upvoteCount;  // number - total upvotes received
  voterSocketIds; // Set<string> - server-side only, never broadcast
}

// AI-generated cluster
class Cluster {
```

```

id;                // string - server-generated UUID
representativeQuery; // string - AI-condensed question
submissionCount;    // number - approx contributing questions
originalQuestions;  // string[] - grouped question texts
}

```

Key Data Decisions

Anonymity by omission

Submissions store no identifying information. The `socketId` field on `Submission` is held only transiently so that a participant can delete their own submission during the session; it is never logged, never sent to the AI, and never included in any response to other clients. When the session ends, all data is discarded.

Original questions stored within clusters

Each `Cluster` object includes the list of original questions that were grouped into it. This allows the Participant View to show the clustered representative query alongside the individual submissions that contributed to it, giving participants transparency into how their question was categorized. This directly addresses the instructor feedback requesting a Participant View where users can see the clustering and provide feedback.

Session tags as clustering context

The host can attach tags to the session (FR1.7) that provide the AI with context about the session's topic. These tags are passed as part of the clustering prompt and improve the quality of the semantic grouping. They are stored on the `Session` object and are not sent to participants.

No database

The current version uses no database. All state is in-memory and ephemeral. This is a deliberate constraint that simplifies the system and aligns with the requirements (NFR5). The tradeoff is that cross-device session access is not supported, a host cannot create a session on one device and view it on another. The Key Decision Log covers the reasoning behind this choice.

Detailed Design

REST API Endpoints

Method	Endpoint	Body	Description
POST	<code>/api/sessions</code>	<code>{ tags?: string[] }</code>	Creates a new session. Returns the session code.
POST	<code>/api/sessions/:code/join</code>	<code>{}</code>	Joins an existing session as a participant.

POST	/api/sessions/:code/submit	{ text: string }	Submits a question. Validates non-empty, char limit, window open.
DELETE	/api/sessions/:code/submit/:id	{ }	Deletes a submission. Validates requester owns it (via socketId).
POST	/api/sessions/:code/close	{ }	Closes the submission window. Host-only.
POST	/api/sessions/:code/cluster	{ }	Triggers AI clustering. Host-only. After close only.
DELETE	/api/sessions/:code/clusters/:id	{ }	Deletes a cluster from the results. Host-only (FR4.7).
POST	/api/sessions/:code/end	{ }	Ends the session. Discards all data. Host-only.

WebSocket Events

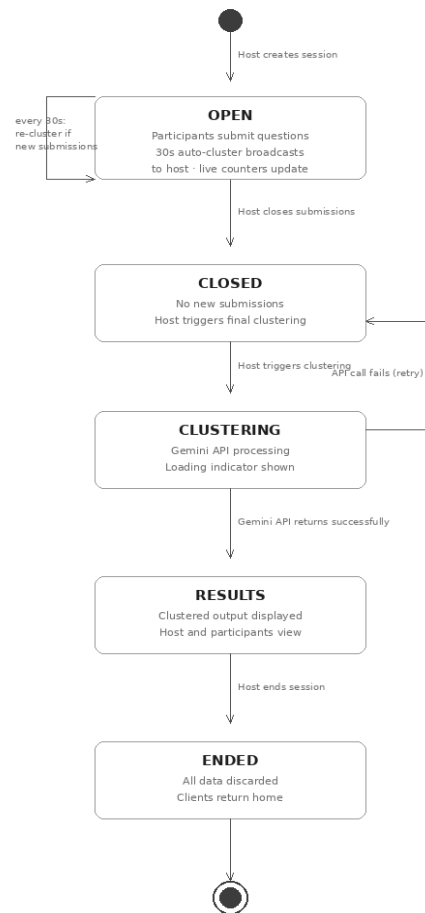
Server to Client (Broadcast)

Event	Payload	Description
participant:joined	{ participantCount }	A new participant connected to the session.
participant:left	{ participantCount }	A participant disconnected.
session:closed	{ }	The host closed the submission window.
session:clustering	{ }	Clustering is in progress. Clients show a loading indicator.
session:results	{ clusters: Cluster[] }	Clustering complete. Results are broadcast to all clients.
session:ended	{ }	The session has ended. Clients return to home.
cluster:deleted	{ clusterId }	The host removed a cluster from the results.
submission:count	{ count }	Updated submission count (host-only broadcast).

Client to Server

Event	Payload	Description
join:room	{ code, role }	Client joins the socket room for the session.
leave:room	{ code }	Client leaves the socket room.

Session State Machine



The session progresses through five phases: OPEN, CLOSED, CLUSTERING, RESULTS, and ENDED. In the OPEN phase the server runs a 30-second periodic re-cluster loop that broadcasts live results to the host. A retry branch from CLUSTERING back to CLOSED handles API failures.

From	To	Trigger
—	OPEN	Host creates a new session via POST /api/sessions.
OPEN	CLOSED	Host closes the submission window via POST /api/sessions/:code/close.
CLOSED	CLUSTERING	Host triggers clustering via POST /api/sessions/:code/cluster.
CLUSTERING	RESULTS	Gemini API returns successfully. Results are broadcast.
CLUSTERING	CLOSED	Gemini API call fails. Host is notified and may retry (FR4.5).
RESULTS	ENDED	Host ends the session via POST /api/sessions/:code/end.

Clustering Flow

1. Periodic re-clustering (OPEN phase): While the submission window is open, the SessionManager runs an interval timer. Every 30 seconds, if at least one new submission has arrived since the last pass, the ClusteringEngine is invoked automatically. The resulting clusters are broadcast to the host via session:results. Participants do not receive these intermediate results.
2. Periodic re-clustering (OPEN phase): While the submission window is open, the SessionManager runs an interval timer. Every 30 seconds, if at least one new submission has arrived since the last pass, the ClusteringEngine is invoked automatically. The resulting clusters are broadcast to the host via session:results. Participants do not receive these intermediate results.
3. Final clustering (CLOSED phase): The host closes the submission window. The server transitions the session phase to CLOSED and immediately triggers a final clustering pass.
4. The server validates the clustering request: session exists, caller is the host, phase is CLOSED, and at least one submission exists (FR4.6). If no submissions exist, the host is notified and no API call is made.
5. The server transitions the session phase to CLUSTERING and broadcasts session:clustering to all connected clients.
6. The ClusteringEngine constructs the prompt. The prompt includes all submitted question texts, the session tags (if any), and instructions to group semantically similar questions, generate a representative query per group, and return the result as structured JSON.
7. The ClusteringEngine calls the Gemini 2.5 Flash API via @google/genai with the constructed prompt.

Submission Deletion Flow

8. The participant clicks the delete icon next to their submitted question. The frontend sends DELETE /api/sessions/:code/submit/:id.
9. The server looks up the submission by ID and verifies that the requesting client's socket ID matches the socketId stored on the submission.
10. If matched, the submission is removed from the session's submission list. The server emits submission:count to update the host's counter.
11. If not matched (or submission not found), the server returns a 403/404 error.

User Experience

Screens

Home Screen

The landing page for all users. It presents two clear paths: "Create a Session" for hosts and "Join a Session" for participants. No account or login is required. The Join path includes a text input for the session code.

Host Dashboard

The primary screen for the host after creating a session. This screen adapts based on the current session phase:

- **OPEN phase:** The session code is displayed prominently (large, centered, easy to project). A QR code is shown beside it for mobile participants. Below, a live participant counter updates in real time via WebSocket. A live submission counter shows how many questions have been received. The host sees a “Close Submissions” button and an area to add or remove session tags.
- **CLOSED phase:** The submission counter is finalized. The host sees a “Cluster Questions” button. If no questions were submitted, the button is disabled and a message explains why (FR4.6).
- **CLUSTERING phase:** A processing indicator is displayed while the AI processes the questions (FR4.2). The “Cluster Questions” button is replaced by a loading spinner.
- **RESULTS phase:** The clustered output is displayed as a list of ClusterCards. Each card shows the representative query, the approximate submission count, and an expandable section showing the original questions that were grouped. The host has a delete button on each cluster (FR4.7). A summary section at the top shows the total number of submissions, the number of clusters, and the session tags. The host sees an “End Session” button.

Participant: Join Screen

After selecting “Join a Session” from the Home Screen, the participant enters the session code and clicks “Join.” The system validates the code. If the code is invalid or the session does not exist, a clear error message is shown (FR2.2). If the submission window is already closed, the participant is informed (FR2.3). On success, the participant is taken to the Participant Session screen.

Participant: Session Screen

The participant’s view of an active session. This screen adapts based on the session phase:

- **OPEN phase:** A text input area with a character counter allows the participant to compose and submit a question. A “Submit” button sends the question. After submission, a confirmation message appears (FR3.7) and the submitted question is shown in a “Your Submissions” list below the input. Each submission in the list has a delete button (FR3.8). The participant can submit additional questions.
- **CLOSED phase:** The text input is disabled. A message informs the participant that submissions are closed and clustering is pending.
- **CLUSTERING phase:** A processing indicator is shown.
- **RESULTS phase:** The clustered output is displayed identically to the host’s view: a list of ClusterCards with representative queries, submission counts, and the original questions listed beneath each cluster. This Participant View allows participants to see how their question was categorized and verify the clustering accuracy.
- **ENDED phase:** A message informs the participant that the session has ended. A button returns them to the Home Screen.

AI Clustering Results Screen

This is the shared results view rendered within both the Host Dashboard and the Participant Session screen once clustering completes. Each cluster is displayed as a card containing the AI-generated representative query as the heading, a badge showing the approximate number of

contributing submissions, and an expandable list of the original user-submitted questions. On the host's view, each card includes a delete button. On the participant's view, the cards are read-only.

Host Writing Answer

The Host selects a specific cluster to address, opening a response section within the cluster card. The text input area allows the Host to type an answer to the cluster while being able to see all the questions in the cluster. There is a "Submit Answer" button and a "Cancel" button for the Host to submit their answer or cancel if they change their mind.

Host Submitting Answer

Upon clicking the "Submit Answer" button, the system updates the state and the Host's response is synchronized across all active Participants. The cluster card turns green, indicating to the Host that his response is successful. The Host also has an option to edit their response which will be synchronized and available to all Participants.

Participant View Post Cluster

This is the participants view after the Host clusters the session. The Participant is shown a live updating list of cluster cards where the Participant can view individual questions asked. The Participant can easily identify their question in the cluster by looking for a blue highlighted answer.

Participant View Post Cluster (Cluster Answered)

Once the host submits a response to a cluster, the corresponding cluster card on the Participants view updates immediately to indicate that the cluster has been answered. The Host's answer is displayed inside of the cluster card right above the individual questions. The answered cluster will turn green to indicate that it has been answered allowing Participants to easily identify the cluster that was answered by the Host.

Participant Question Upvoting

Each ClusterCard in the participant view includes upvote buttons on the individual questions listed beneath the representative query. A participant can upvote any question once. The upvote count for each question is displayed next to the question text and updates in real time via WebSocket. The host view shows the same upvote counts, giving the host a secondary signal of collective priority beyond cluster size alone. A participant cannot upvote their own question.

Session Summary Screen

Displayed to both the host and all active participants when the host ends the session. The summary lists each cluster's representative query, the number of submissions that contributed to it, the host's written answer (if one was submitted), and the upvote totals for the questions within that cluster. The screen includes a "Copy Summary" button that copies the full structured output to the clipboard so the host can paste it into a follow-up document or message.

Wireframes

Home Screen

Popular Vote

Collect and Cluster Questions Anonymously



Create Session

Start a new session and get a code to share with participants

Create new session



Join Session

Enter a session code to participate and submit questions

ENTER SESSION CODE

Join session

Host Dashboard (OPEN Phase)

Host Dashboard

Manage your session and view submissions

Session Code

Share this code with participants

JN45GCY



Copy Code

Live Q+A Session

Questions are being submitted in real-time. Click "Start AI Clustering" to update question clustering as new questions arrive.

Session Info

Participants 5

Submissions 5

Status Open

Session Tags

Add session tags to help the AI understand the session topic

Technology, Education, Business

Education Math

Session Controls

Close Session

Start AI Clustering

Participant Session (OPEN Phase)

Participant View

Submit questions and view live clusters

Submit Question

Type your question here...

0 / 500

Submit

Your Stats

Submitted1

In Cluster0

Your Questions

is the test this week or next week

Session Code
JN45GCY

Live Question Clustering

Similar questions are grouped together. Your questions are highlighted in blue.

Clustering Results View

Host Dashboard

Manage your session and view submissions

Session Code

Share this code with participants

JN45GCY

Copy Code

Session Info

Participants5

Submissions5

StatusOpen

Session Tags

Add session tags to help the AI understand the session topic

Technology, Education, Business

EducationMath

Session Controls

Close Session

Start AI Clustering

Live Q+A Session

Questions are being submitted in real-time. Click "Start AI Clustering" to update question clustering as new questions arrive.

Date and timing of the upcoming exam?

3 questions

Answer

Individual questions in this cluster

When is the actual exam? I heard it was Friday.

is the test this week or next week

What day is the test?

Resources and tools allowed during the test (calculators/formula sheets)

2 questions

Answer

Individual questions in this cluster

Will there be a formula sheet provided for the law of exponents?

can we use calculators on the exam on friday?

Host Writing Answer

Host Dashboard

Manage your session and view submissions

Session Code

Share this code with participants

JN45GCV



Copy Code

Session Info

Participants 5

Submissions 5

Status Open

Session Tags

Add session tags to help the AI understand the session topic

e.g. Technology, Education, Business

Education Math

Session Controls

Close Session

Start AI Clustering

Live Q+A Session

Questions are being submitted in real-time. Click "Start AI Clustering" to update question clustering as new questions arrive.

Date and timing of the upcoming exam?

3 questions

Answer

Your Answer

The exam is March 18th, 11:30 AM

Submit Answer

Cancel

Individual questions in this cluster

When is the actual exam? I heard it was Friday.

Is the test this week or next week

What day is the test?

Resources and tools allowed during the test (calculators/formula sheets)

2 questions

Answer

Individual questions in this cluster

Will there be a formula sheet provided for the law of exponents?

Can we use calculators on the exam on Friday?

Host Submitting Answer

Host Dashboard

Manage your session and view submissions

Session Code

Share this code with participants

JN45GCV



Copy Code

Session Info

Participants 5

Submissions 5

Status Open

Session Tags

Add session tags to help the AI understand the session topic

e.g. Technology, Education, Business

Education Math

Session Controls

Close Session

Start AI Clustering

Live Q+A Session

Questions are being submitted in real-time. Click "Start AI Clustering" to update question clustering as new questions arrive.

Date and timing of the upcoming exam?

3 questions

Answered

Answer

Answer

The exam is March 18th, 11:30 AM

edit

Individual questions in this cluster

When is the actual exam? I heard it was Friday.

Is the test this week or next week

What day is the test?

Resources and tools allowed during the test (calculators/formula sheets)

2 questions

Answer

Individual questions in this cluster

Will there be a formula sheet provided for the law of exponents?

Can we use calculators on the exam on Friday?

Participant View Post Cluster

Participant View

Submit questions and view live clusters

Submit Question

Type your question here...

0 / 500

Submit

Your Stats

Submitted 2

In Cluster 1

Your Questions

is the test this week or next week

When are you grading the first exam?

Live Question Clustering

Similar questions are grouped together. Your questions are highlighted in blue.

Date and timing of the upcoming exam?

3 questions

Individual questions in this cluster

When is the actual exam? I heard it was Friday.

is the test this week or next week Your Question

What day is the test?

Resources and tools allowed during the test (calculators/formula sheets)

2 questions

Individual questions in this cluster

Will there be a formula sheet provided for the law of exponents?

can we use calculators on the exam on friday?

Participant View Post Cluster (Cluster Answered)

Participant View

Submit questions and view live clusters

Submit Question

Type your question here...

0 / 500

Submit

Your Stats

Submitted 2

In Cluster 1

Your Questions

is the test this week or next week

When are you grading the first exam?

Live Question Clustering

Similar questions are grouped together. Your questions are highlighted in blue.

Date and timing of the upcoming exam?

3 questions **Answered**

Answer

The exam is March 18th, 11:30 AM

Individual questions in this cluster

When is the actual exam? I heard it was Friday.

is the test this week or next week Your Question

What day is the test?

Resources and tools allowed during the test (calculators/formula sheets)

2 questions

Individual questions in this cluster

Will there be a formula sheet provided for the law of exponents?

can we use calculators on the exam on friday?

Participant Question Upvoting

Participant View

Submit questions and view live clusters

Session Code
JN45GCV

Submit Question

Type your question here...

0 / 500

Submit

Your Stats

Submitted 2

In Cluster 1

Your Questions

is the test this week or next week ✕

When are you grading the exam? ✕

Live Question Clustering

Similar questions are grouped together. Your questions are highlighted in blue. Upvote questions to signal priority.

Date and timing of the upcoming exam?

3 questions ▲ Upvote to prioritize

Individual questions in this cluster

When is the actual exam? I heard it was Friday. ▲ 4

is the test this week or next week ▲ 7

What day is the test? ▲ 2

Resources and tools allowed during the test (calculators/formula sheets)

2 questions

Individual questions in this cluster

Will there be a formula sheet provided for the law of exponents? ▲ 3

can we use calculators on the exam on friday? ▲ 1

Session Summary Screen

Session Summary

Session ended · 5 clusters · 12 submissions · JN45GCV

5 Clusters

12 Submissions

3 Answered

Copy Summary

Date and timing of the upcoming exam?

3 questions ▲ 13 upvotes Answered

Answer

The exam is March 18th at 11:30 AM

Individual questions in this cluster

When is the actual exam? I heard it was Friday. ▲ 4

is the test this week or next week ▲ 7

Grading timeline for first exam?

2 questions ▲ 9 upvotes Answered

Answer

Graded by end of next week, posted on Blackboard.

Individual questions in this cluster

When are you grading the first exam? ▲ 9

Extra credit or curve policy?

2 questions ▲ 6 upvotes

Individual questions in this cluster

Is there any extra credit available this semester? ▲ 6

Will there be a curve if the class average is low? ▲ 0

Resources allowed during the test?

2 questions ▲ 4 upvotes

Individual questions in this cluster

Will there be a formula sheet for exponents? ▲ 3

Can we use calculators on the exam? ▲ 1

Key Decision Log

Node.js/Express over Python/Flask

Decision: The backend uses Node.js with Express instead of Python with Flask.

Reason: The entire stack uses JavaScript, React on the frontend, Node on the backend, which means any team member can work on any part of the codebase without context-switching languages. Express handles the asynchronous demands of Socket.IO and the Gemini SDK. Socket.IO was built for Node and is its most mature implementation. Node's async-native design handles concurrent WebSocket connections without needing an additional library like eventlet or gevent (which Flask-SocketIO requires for production).

Impact: The team benefits from a single-language stack. The Gemini SDK has an official Node.js package (@google/genai) that works identically to the Python version.

In-memory state over a database

Decision: All session data is stored in server memory with no database.

Reason: The requirements explicitly state that no data should be retained after a session ends (NFR5). A database would add deployment complexity, schema management, and an obligation to handle data cleanup, all for data that is intentionally ephemeral. In-memory storage is the simplest solution that satisfies the requirements.

Impact: Cross-device session access is not supported. If the server restarts, all active sessions are lost. These are the accepted tradeoffs of an in-memory design, and they align with the requirements' explicit prohibition on data retention.

Socket.IO over raw WebSocket

Decision: Socket.IO is used for real-time communication instead of the raw WebSocket API.

Reason: Socket.IO provides automatic reconnection, room-based message routing (which maps naturally to sessions), and fallback to HTTP long-polling if WebSocket is unavailable. These features would need to be implemented manually with raw WebSockets and would consume significant development time for a course project.

Impact: Socket.IO adds ~40KB to the client bundle. The room abstraction simplifies broadcasting to all participants in a session.

Participant View shows clusters with original questions

Decision: When clustering results are released, participants see the same cluster view as the host, including the original user-submitted questions listed under each cluster.

Reason: This directly addresses the instructor feedback recommending a Participant View that shows the clustered version of the question alongside the user-submitted questions beneath it. By showing participants how their questions were grouped, the system creates a feedback loop: if a participant disagrees with the clustering, they can flag it to the host verbally, which improves the overall quality of the session.

Impact: The cluster broadcast payload must include the original question texts (not just the representative query and count). This increases the payload size but is bounded by the number of submissions in a session.

Clustering runs on two triggers: periodic during open, final on close

Decision: Clustering runs on two triggers. While the submission window is open, the server automatically re-clusters every 30 seconds if new submissions have arrived since the last pass, broadcasting live ClusterCards to the host. When the host closes the window, a final clustering pass runs before results are released to participants.

Reason: Triggering a cluster pass on every individual submission produces unstable results, cluster boundaries shift with each new question. Two triggers balance responsiveness with stability: the 30-second interval gives the host a live view of emerging themes while the window is open, and the final pass on close gives participants a stable, complete result. This also stays within Gemini's free tier limit of 10 RPM.

Impact: The host sees live ClusterCards updating every 30 seconds while the submission window is open. Participants see the final stable clustering once the host closes the window and releases results.

Google Gemini over OpenAI

Decision: Clustering is performed by calling the Google Gemini API (Gemini 2.5 Flash) rather than the OpenAI API.

Reason: Gemini offers a free tier requiring no credit card, with 10 requests per minute and 250 requests per day. Since clustering is a single API call per session, this free tier is more than sufficient for development, testing, and live demos. OpenAI does not offer a free tier—every API call requires payment. For a course project with no budget, the free tier is decisive. Gemini 2.5 Flash also supports structured JSON output, simplifying response parsing.

Impact: The system depends on the Gemini API being available (CON1). An API key must be configured on the server. The ClusteringEngine is designed to be swappable, so an alternative provider could replace Gemini if needed.

No authentication

Decision: Neither hosts nor participants are required to create accounts or log in.

Reason: The system is designed for ephemeral, single-use sessions. Adding authentication would increase the barrier to entry for participants (who are expected to join quickly during a live event) and add implementation complexity. The session code itself acts as a lightweight access control mechanism (NFR2).

Impact: The host cannot resume a session from a different device. A single participant could submit many questions, which is mitigated by rate limiting.

Host identified by socket connection, not credentials

Decision: The host is identified by the socket connection that created the session. Host-only actions (close submissions, trigger clustering, delete cluster, end session) are validated by checking the requesting socket ID against the session's hostSocketId.

Reason: Without user accounts, the socket connection is the most reliable way to identify the host. It is simple to implement and sufficient for the single-device, single-session model the system targets.

Impact: If the host's browser tab is closed, Socket.IO's automatic reconnection will attempt to re-establish the connection. If reconnection fails, the session becomes unrecoverable. This is an accepted consequence of the stateless, no-persistence design.

Participants can add a submission directly to an existing cluster

Decision: Each ClusterCard in the participant view includes an “Add my question here” button. Clicking it opens the standard question input, but tags the submission with the target cluster ID. The server appends the submission directly to that cluster’s question list and issues a targeted Gemini call to regenerate the representative query for that cluster only. The updated cluster is broadcast to all connected clients.

Reason: Freeform AI clustering will not always group questions the way a participant intended. Giving participants the ability to place a question directly into a cluster they identify as the right one improves clustering accuracy without requiring the host to intervene. It also closes the feedback loop from the Participant View: rather than only seeing where their question was placed, participants can correct it.

Impact: A new REST endpoint (POST /api/sessions/:code/clusters/:clusterId/submit) handles cluster-targeted submissions. The SubmissionHandler is extended to accept an optional clusterId field. The ClusteringEngine exposes a regenerateRepresentativeQuery method that re-prompts Gemini with the updated question list for a single cluster. The Cluster data model is unchanged; the question list grows and the representative query is updated in place.

Question upvoting surfaces collective priority within clusters

Decision: Participants can upvote individual questions listed within a ClusterCard. Each participant has one upvote per question. Upvote counts update in real time via WebSocket and are visible to both participants and the host.

Reason: Cluster size alone is a coarse signal—a cluster of ten questions does not indicate which of those questions the group considers most important. Upvoting gives participants a way to express priority within a cluster without breaking anonymity. It also gives the host a clear signal of where to focus responses first.

Impact: The Submission data model gains a upvoteCount field and a voterSocketIds set (server-side only, never broadcast) to enforce the one-upvote-per-participant rule. A new WebSocket event (cluster:upvote) broadcasts the updated question upvote count to all clients in the session room.

Session summary consolidates clustering results and host responses for export

Decision: When the host ends the session, the system displays a structured summary screen to all connected clients. The summary includes each cluster’s representative query, submission count, upvote totals, and the host’s written answer if one was provided. A copy-to-clipboard button lets the host export the summary without requiring file storage or accounts.

Reason: Sessions are ephemeral by design, no data is retained after the session ends. A clipboard export gives the host a lightweight path to preserve the output without requiring the system to introduce persistence, accounts, or a download mechanism. It satisfies the practical need to follow up on session results while keeping the architecture simple.

Impact: The summary screen is a read-only view rendered from the session’s in-memory cluster and answer state. The copy action serialises the cluster list into plain text on the client side using the Clipboard API. No new server endpoints or storage are required.